

- (a) Consider the Voronoi diagram of the center points of the atoms of  $P$ . Show that an atom  $p_i \in P$  is solvent-accessible if and only if there exists a point within  $p_i$ 's Voronoi cell that is at distance at least  $r + r'$  from  $p_i$ .

**Solution:** Let's consider a water molecule at point  $q$  in the Voronoi cell of  $p_i$  such that the distance between  $q$  and  $p_i$  is at least  $r + r'$ . Given that  $q$  is in the Voronoi cell of  $p_i$ , we know that  $q$  is closer to  $p_i$  than any other atom in  $P$ . Therefore every other atom in  $P$  is at least  $r + r'$  away from  $q$ . If we draw a circle of radius  $r'$  around  $q$  and circles of radius  $r$  around all points in  $P$ , then the water molecule will not overlap with any atoms in  $P$ . Therefore,  $q$  is solvent-accessible.

No point in the Voronoi cell of  $p_i$  that is less than  $r + r'$  away from  $p_i$  is solvent-accessible, and every point in the Voronoi cell of  $p_i$  that is at least  $r + r'$  away from  $p_i$  is solvent-accessible. Therefore, an atom  $p_i \in P$  is solvent-accessible if and only if there exists a point within  $p_i$ 's Voronoi cell that is at distance at least  $r + r'$  from  $p_i$ . ■

- (b) Using (a), show that it is possible to determine all the atoms that are solvent-accessible in  $O(n \log n)$  time. [Hint: Don't forget that some Voronoi cells can be unbounded.]

**Solution:** To determine all the atoms that are solvent-accessible, we first compute the Voronoi diagram for  $P$  in  $O(n \log n)$ . We begin by designating all unbounded Voronoi cells as solvent-accessible. Now we consider all other Voronoi cells. For each Voronoi cell, we compute the distance from the center point to each of the cell's vertices. The vertices are guaranteed to be the farthest points from the center. If the distance to any single vertex is at least  $r + r'$ , then the Voronoi cell is solvent-accessible. We iterate over all Voronoi cells and vertices in  $O(n)$  time. Therefore, we can determine all the atoms that are solvent-accessible in  $O(n \log n)$  time. ■

- (a) Recall, each segment endpoint  $a_i$  or  $b_i$  has a corresponding dual line ( $a_i^*$  or  $b_i^*$ ), and the line  $\ell$ , if it exists, has a dual point  $\ell^*$ . In terms of these dual objects, describe an equivalent situation to  $\ell$  intersecting all the line segments of  $S$ .

**Solution:** If we assume that  $a_i$  is above  $b_i$ , then, for  $\ell$  to intersect all segments in  $S$ , all  $a_i$  must be above  $\ell$  and all  $b_i$  must be below. In the dual plane, the equivalent situation is that the dual point  $\ell^*$  is above all  $a_i^*$  and below all  $b_i^*$ . ■

- (b) Describe an  $O(n^2)$  time algorithm to determine if there does exist a non-vertical line  $\ell$  that intersects all of the segments of  $S$ . If your algorithm involves an  $O(n^2 \log n)$  time plane sweep over an arrangement of  $O(n)$  lines, then you may assume there exists a topological plane sweep variant that runs in time  $O(n^2)$ .

**Solution:** To begin, we convert the segment endpoints in  $S$  to dual lines and find the line arrangement for the dual lines in  $O(n^2)$ . We then sort the dual lines by the  $x$ -value of their primal points. This ensures the lines are ordered by their  $y$ -value at  $x = -\infty$ . We then sweep a vertical line from left to right. As we sweep, we maintain the order in which the dual lines intersect the sweep line.

Between each pair of adjacent dual lines, there will be a region of the line arrangement. If a point,  $\ell^*$ , in the region is bounded by  $a_i^*$  and  $b_i^*$  of a given segment, as described in part (a), then every point in that region is also bounded by  $a_i^*$  and  $b_i^*$ . Therefore, the primal line  $\ell$  intersects the segment for all points in this region. Based on this rule, we will maintain a count of the number of segments that intersect  $\ell$  as we sweep. We initialize this count as follows:

- We traverse the dual lines from top to bottom. The region above all the lines begins with a count of 0.
- Whenever we encounter a line  $a_i^*$ , we add 1 to the count of the previous region and apply the new count to the next region.
- Whenever we encounter a line  $b_i^*$ , we subtract 1 from the count of the previous region and apply the new count to the next region.

This will run in  $O(n)$  time.

Now we begin our sweep. As we sweep, we will update the count based on the regions we encounter. If the count ever reaches  $n$ , then we have found a line  $\ell$  that intersects all segments in  $S$ . If the count never reaches  $n$ , then no such line exists. Every time we come to an intersection, we swap the intersecting lines in our list. The regions above and below the intersection will maintain the same count. We'll have to calculate the count for the new region to the right of the intersection using the count of the previous region to the left. There are three cases to consider:

1. If the previous region is bounded by  $a_i^*$  on top and  $b_j^*$  on bottom, (where  $i$  may or may not equal  $j$ ), then we are exiting the regions bounded by  $a_i^*$  and exiting the regions bounded by  $b_j^*$ . Therefore, we subtract 2 from the count.
2. If the previous region is bounded by  $b_j^*$  on top and  $a_i^*$  on bottom, (where  $i$  may or may not equal  $j$ ), then we are entering the regions bounded by  $a_i^*$  and entering the regions bounded by  $b_j^*$ . Therefore, we add 2 to the count.
3. If the previous region is bounded by  $a_i^*$  and  $a_j^*$  or  $b_i^*$  and  $b_j^*$ , then we are exiting one of these regions and entering the other. Therefore, we add and subtract 1 from the count which cancels out.

We iterate over every intersection in the line arrangement in  $O(n^2)$  time. Therefore, we can determine if there exists a line  $\ell$  that intersects all segments of  $S$  in  $O(n^2)$  time.

■